# Guía de Ciberseguridad: Auditoría IA

Las Inteligencias Artificiales tienen un objetivo principal: **Hacer que el código funcione a toda costa**. Para lograr esto, la IA frecuentemente tomará el "camino más fácil", saltándose barreras de seguridad. Como Arquitecto, la ciberseguridad es tu responsabilidad número uno.

## 1. El Ataque de la Consola (Exposición de la DB)

En tecnologías como Supabase o Firebase, la base de datos se puede consumir directamente desde React. Esto es útil, pero extremadamente peligroso.

**El Escenario de Terror (Vibe Coding):**
Le pides a la IA que haga un sistema para listar usuarios. Un atacante abre DevTools y escribe:

```
// El atacante ve tu cliente de Supabase expuesto:
const { data } = await supabase.from('users').select('*');
console.log(data); // ¡Acaba de descargar toda tu BD!
```

**Cómo auditarlo (La Solución):**
Debes exigirle a la IA y aplicar las **Row Level Security (RLS)**.

> *Instrucción: "Asegúrate de escribirme los comandos SQL para activar RLS en la tabla `users` para que solo el usuario autenticado pueda ver su propia información."*

## 2. Fuga de API Keys (El "God Mode" Expuesto)

La IA es famosa por escribir contraseñas directamente en el código o usar las llaves equivocadas.

- Supabase te da dos llaves: anon_key (Pública) y `service_role_key` (Modo Dios).
- **El Problema:** Para solucionar problemas de permisos rápido, la IA te sugerirá usar la `service_role_key` en React. Si lo haces, cualquiera tendrá control total sobre tu infraestructura.

**La Regla de Oro:**

- NUNCA subas una API Key directamente al código de React.
- Obliga a la IA a usar variables de entorno (`.env`).
- Las llaves de administrador deben ir **exclusivamente** en las funciones de tu Backend.

## 3. Confianza Ciega en el Frontend

La IA es excelente creando validaciones visuales, pero a nivel de seguridad, **el frontend no existe**.

**El Ataque:** Un atacante ignora tu web y envía peticiones directamente a tu Backend mediante Postman. Si tu Backend no valida los datos asumiendo que "el frontend ya lo hizo", tu sistema caerá.

> *Instrucción: "Voy a crear un endpoint en el backend. NO asumas que los datos vienen limpios. Escribe un middleware o usa Zod para validar estrictamente todos los campos entrantes."*

## 4. Comunicación Server-to-Server (S2S)

Para operaciones críticas (ej. pagos), la IA suele cometer el error de escribir la llamada a la API externa directamente en React. Esto expone la llave secreta de esa API.

**La Solución (Arquitectura S2S):**
El navegador jamás debe hablar directamente con servicios externos críticos.

1. **Cliente → Servidor:** React hace una petición limpia a TU Backend.
2. **Servidor → Servidor:** Tu Backend toma su propia llave de seguridad y se comunica con el servicio externo.
3. **Respuesta:** Tu Backend valida y le pasa solo el resultado necesario de vuelta a React.

> *Instrucción: "Necesito conectarme a una API externa. Por seguridad, no lo hagas desde el Frontend. Créame una Edge Function para que actúe como intermediario."*

## 5. Control de Costos y Rate Limiting (FinOps)

En la nube, la seguridad no solo protege tus datos, también protege tu billetera.

**El Escenario de Terror:**
Implementaste S2S perfectamente. Sin embargo, un atacante descubre el endpoint y envía 10,000 peticiones por segundo a OpenAI. La llave no fue robada, pero el atacante usó tu propia infraestructura para generarte una factura de miles de dólares.

**La Solución (Defensa de Infraestructura):**

- **Rate Limiting:** Instruye a tu backend a bloquear múltiples peticiones de una misma IP.
- **Caché:** Guarda respuestas costosas para no repetir llamadas de IA.

## Conclusión

*La regla fundamental de la ciberseguridad asistida por IA es: **Paranoia Absoluta**. Asume siempre que la IA intentará dejar puertas abiertas. Usa PCA para predecir estos vectores de ataque e imponer la seguridad desde el Scaffolding.*